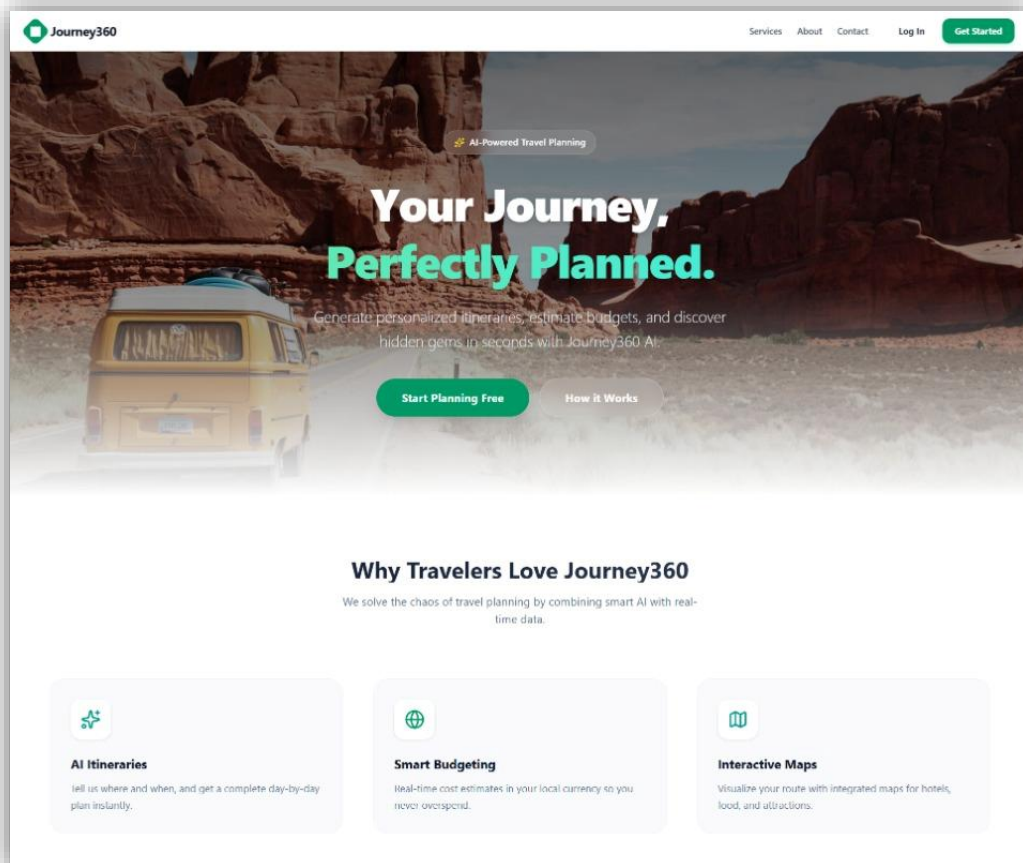


Journey360 – Final Technical Report

Journey360 – Your Smart Travel Companion



Submitted by,

Team: IXR_Prismatics

Gowtham G

Aaron Alphons Thomas

Kaarthik M

Sandhiya U

1. Executive Summary

Journey360 is our attempt to make travel feel simpler, safer, and more human. As travelers ourselves, we noticed that planning and navigating a trip today usually means juggling multiple apps—one for itineraries, one for maps, another for translation, and yet another for safety. While each tool solves a small problem, together they create stress, confusion, and constant context switching.

We built Journey360 to remove that friction. Instead of forcing users to adapt to technology, Journey360 adapts to the traveler. From planning a trip, to navigating unfamiliar streets using AR, to staying informed about safety conditions, everything lives inside one unified platform. The goal is not just efficiency, but confidence—helping users feel prepared and supported wherever they go.

Artificial Intelligence plays a supportive role in this system. AI helps generate itineraries, assist users through chat, and summarize safety information, but it never replaces human control. All workflows, logic, and system behavior were carefully designed, reviewed, and validated by our team. AI acts as an assistant, not a decision-maker.

Journey360 aligns strongly with Samsung PRISM values by focusing on real-world problems, ethical AI usage, and system-level innovation. It demonstrates how thoughtful engineering and responsible AI can work together to improve everyday experiences.

2. Introduction & Problem Statement

Travel has become easier to access, but harder to manage. A single trip often requires switching between multiple applications—for planning routes, finding places, translating languages, checking safety updates, and managing bookings. Each app demands attention, and together they increase cognitive load, especially in unfamiliar environments.

This becomes most visible during real travel moments—when a traveler is walking through a new city, trying to understand directions, communicate with locals, or respond to unexpected situations. Existing solutions focus on individual features rather than the traveler’s overall experience. Safety is often treated as an add-on, and context-awareness is limited.

Journey360 was created to address this gap. Instead of solving isolated problems, we focused on the full travel journey. Our aim was to build a system that quietly supports users in the background—anticipating needs, reducing friction, and making travel feel more manageable and less stressful.

System Architecture & Design

Journey360 is built with a clean, modular architecture that supports real-time interaction, immersive navigation, and safety-critical workflows. Every architectural decision was made with reliability, clarity, and future scalability in mind.

Architecture Explanation

The system is organized into **three main layers**, each with a clear responsibility.

1. Presentation Layer (Frontend)

The frontend is built using **React (v18+)** with **Vite**, styled with **Tailwind CSS** and **custom CSS**. This layer is what users directly interact with, and it was designed to stay simple even when the system underneath is complex.

It handles:

1. Trip input and itinerary visualization
2. Interactive maps using **Leaflet**
3. AI Travel Assistant chat interface
4. AR navigation visuals rendered with **Three.js**
5. Safety alerts and one-tap SOS access
6. Trip dashboards and saved places

Special care was taken to reduce cognitive load. Important actions—like navigation and safety—are always easy to reach, and the interface avoids overwhelming users, especially in stressful situations.

2. Application Layer (Backend Services)

The backend is implemented using **FastAPI (Python)** and acts as the brain of Journey360. It coordinates all core logic and system behavior, including:

1. Smart itinerary generation based on user preferences
2. Context-aware AI responses using Google Gemini
3. Safety score calculation using weather and news data
4. Translation request handling
5. AR navigation coordination
6. Trip management and automated email delivery of itineraries

This layer exposes clean REST APIs to the frontend, ensuring smooth communication while keeping business logic well-structured and maintainable.

3. Data & External Services Layer

This layer manages persistence, authentication, and external intelligence:

1. **MongoDB** stores user profiles, itineraries, saved places, and safety data
2. **Firebase Authentication** ensures secure login with Email/Password, Google Sign-In, and optional 2FA
3. **OpenRouteService** provides routing and navigation paths
4. **Geolocation API** supplies real-time user location
5. **Device Orientation API** enables compass-based AR guidance
6. **OpenWeather API** delivers weather alerts
7. **Location-based safety news API** provides regional safety updates

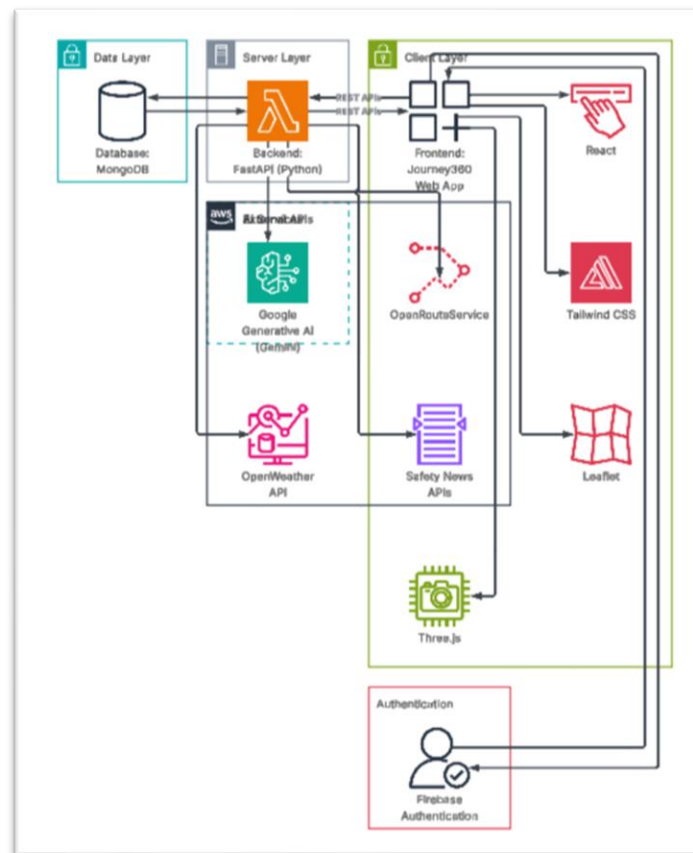
All integrations were manually implemented with proper error handling and fallback mechanisms to ensure reliability even when network conditions are unstable.

Architecture Summary

1. **Frontend:** React + Tailwind CSS + Leaflet + Three.js
2. **Backend:** FastAPI (Python)
3. **Database:** MongoDB
4. **Authentication:** Firebase Authentication
5. **AI Services:** Google Generative AI (Gemini)
6. **External APIs:** OpenRouteService, OpenWeather, Safety News APIs

The frontend communicates with backend services via REST APIs. The backend processes user inputs, coordinates AI assistance, evaluates safety conditions, and returns structured results to the UI. This layered approach allows Journey360 to remain flexible, stable, and ready for future expansion.

Architecture Flow



AI Usage Disclosure Form

1. Team Details

Team Name: **IXR_Prismatics**

Project / Product Name: **Journey360 – Your Smart Travel Companion from Start to Finish**

Organization / Institution : **Vellore Institute of Technology ,Vellore**

Submission Date: **27.01.2026**

2. AI Usage Declaration

Did your team use any AI / Gen AI tools or services in developing or creating project artifacts? **Yes.**

AI tools were used as **assistive support** during ideation, design exploration, development assistance, testing support, and documentation refinement. All core ideas, feature definitions, architectural decisions, and final implementations were **owned, validated, and finalized by the team.**

3. Purpose of AI Usage

Idea generation / brainstorming

The idea for Journey360 began with simple, honest conversations within our team. Each member shared personal travel struggles—using multiple apps to plan trips, feeling unsafe in unfamiliar locations, getting lost, or facing language barriers. These discussions helped us realize how fragmented and stressful modern travel experiences can be.

Through collaborative brainstorming, we asked ourselves what kind of solution we would genuinely trust and rely on while traveling. By reviewing existing travel apps and identifying what they lacked, we shaped Journey360 into a unified platform that connects planning, navigation, safety, and communication. This process directly influenced features such as the **Smart Itinerary Builder**, **AR Navigation**, and **Offline Translation**, ensuring that each feature addresses a real user pain point.

Code generation or assistance

As development progressed, Journey360 evolved into a system with multiple interconnected components—handling user inputs, API integrations, and data flow between frontend and backend services. While the team had clarity on the desired outcomes, determining efficient and maintainable implementation approaches required exploration.

Tools such as **Gemini AI**, **Cursor**, **Antigravity Editor**, and **Claude AI** were used to assist with reasoning through complex logic, exploring API usage patterns, and debugging edge cases. For example, while integrating navigation, translation, and safety-related APIs, these tools helped us identify potential implementation issues early.

All AI-generated suggestions were treated as references only. The team reviewed, modified, and often rewrote outputs to match our system design and standards. This approach ensured that features like **AR Navigation** and **Offline SOS** worked reliably and aligned with our intended user experience.

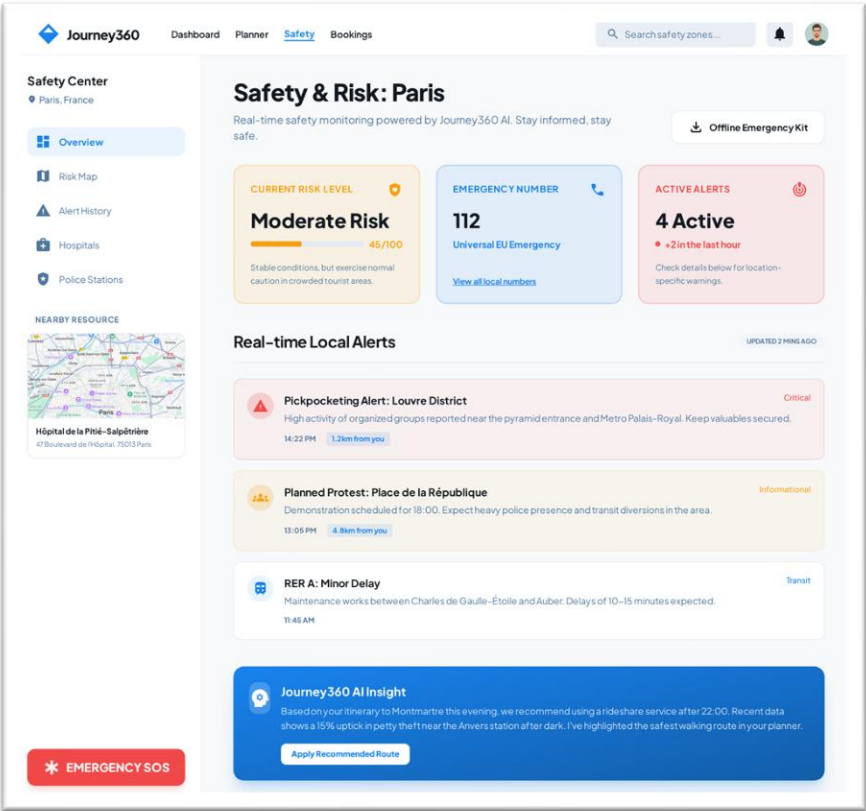
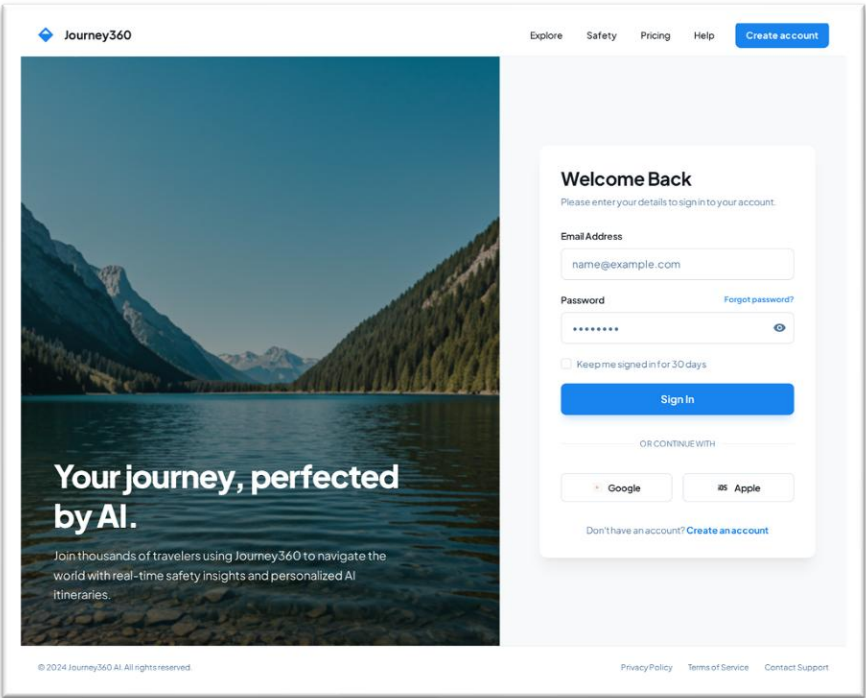
UI / UX design

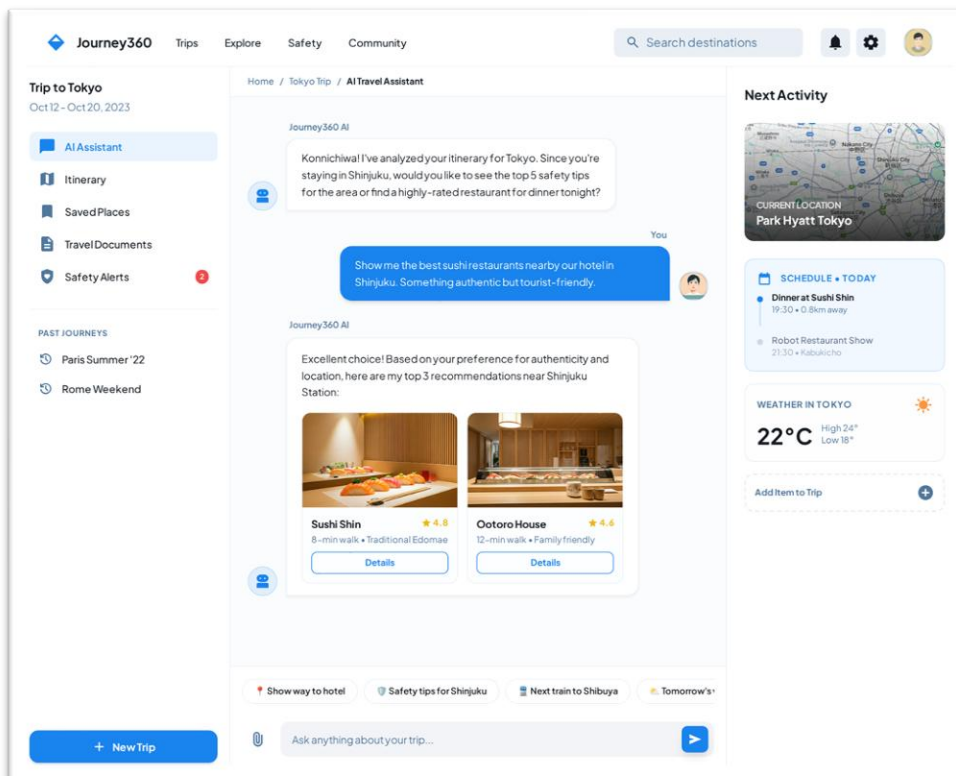
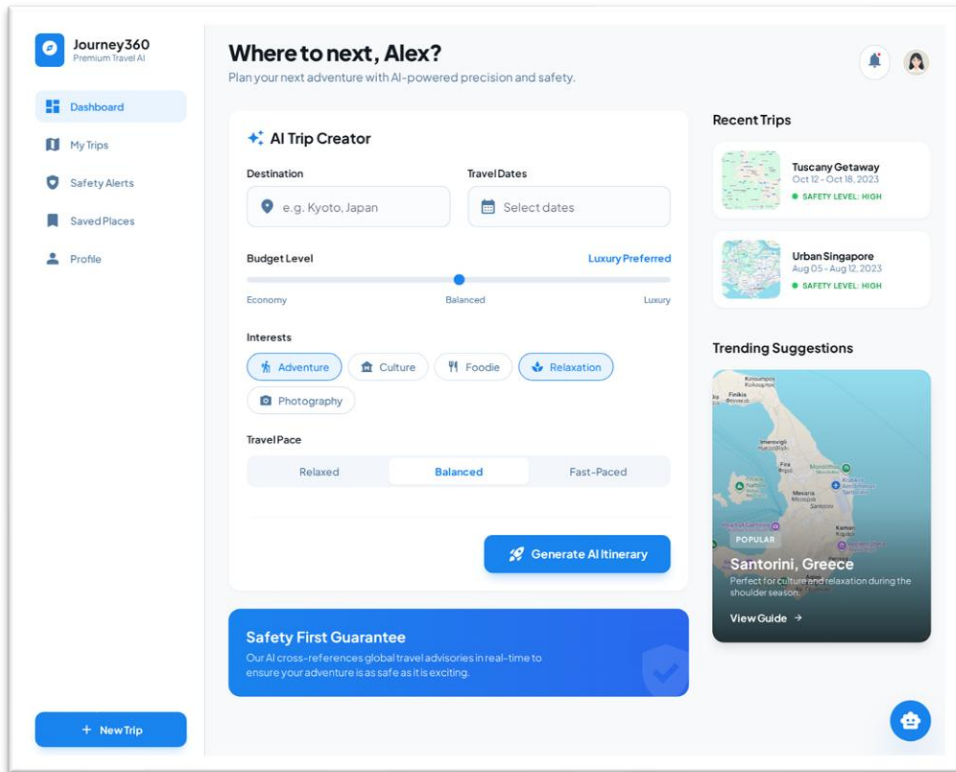
From the start, we wanted Journey360 to feel calm, intuitive, and trustworthy—especially for users traveling in unfamiliar environments. We focused on minimizing cognitive load and making critical features easy to access when users need them most.

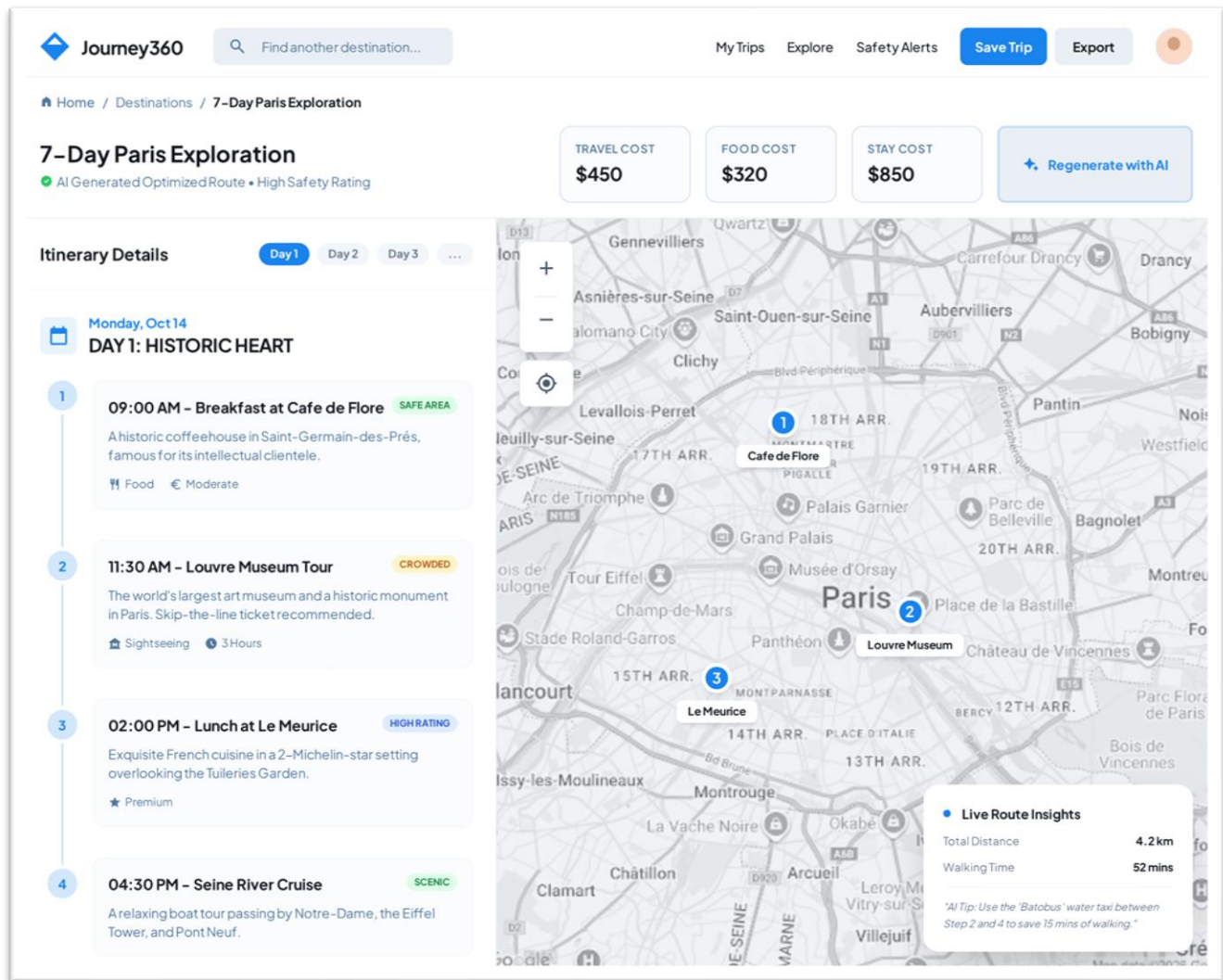
To quickly visualize ideas, we used **Stitch AI** to generate early UI concepts and screen layouts. These helped us compare different flows and understand how users might interact with features such as itinerary planning, AR navigation overlays, and emergency access.

After this exploration phase, all designs were manually refined by the team. Decisions around layout, color, spacing, and interactions were made to ensure clarity and ease of use. As a result, features like **one-tap SOS**, **camera-based AR guidance**, and **clear itinerary views** directly support safer, more confident travel experiences.

Initial design made with stich AI







Content creation

For content creation and documentation, we used **ChatGPT and Gemini** to help refine clarity, tone, and structure across project documents, feature descriptions, and presentation materials. These tools were especially helpful in organizing our thoughts, improving readability, and ensuring our explanations were easy to understand for both technical and non-technical audiences.

All generated content was carefully reviewed, edited, and contextualized by the team. We ensured that the final documentation accurately reflected our own ideas, design decisions, and implementation details, maintaining originality and alignment with the project's objectives.

Data analysis

Not used.

AI tools were not used for data analysis in the development of Journey360. All data handling, logic validation, and evaluation of system behavior were performed manually by the team based on project requirements and API responses.

Testing / debugging

Testing played a critical role in ensuring Journey360 functioned reliably across different scenarios. We used **AI-supported editor tools** to assist with automated test generation, identify edge cases, and flag potential integration issues.

These tools helped us simulate scenarios such as navigation errors, offline translation availability, and emergency trigger handling. All test results were manually reviewed, and final fixes were implemented by the team. This process ensured that key features—especially **safety alerts**, **offline functionality**, and **real-time navigation**—perform consistently and can be trusted by users during travel.

Other

Beyond the main development areas, we also used AI tools in small, supportive ways throughout the project. These tools helped us organize our thoughts, structure documentation, and clearly explain how Journey360 works—from feature flows to system behavior and user journeys.

They were also useful when we reflected on important aspects like user safety, privacy, and responsible technology use, especially since Journey360 is meant to support travelers in unfamiliar and sometimes stressful situations. AI helped us think through how to communicate these ideas clearly and responsibly.

That said, all decisions, interpretations, and final content came from our team. AI tools simply helped us express our ideas better and stay organized while working on a complex, end-to-end travel solution.

4. Feature Origin Classification

4.1 Feature 1

4.1.1 Feature Name: Smart Itinerary Planning

4.1.2 Origin: Both (Self-Generated + AI-Assisted)

4.1.3 Description:

Smart Itinerary Planning is the core intelligence of Journey360. It generates detailed, realistic, day-by-day itineraries based on user inputs such as destination, duration, budget, interests, pace, and local conditions.

While the idea and logic were fully self-generated by the team, Google Generative AI (Gemini) was used as a **controlled content generator**, strictly guided through a structured system prompt.

AI Prompt Usage (Itinerary Generation):

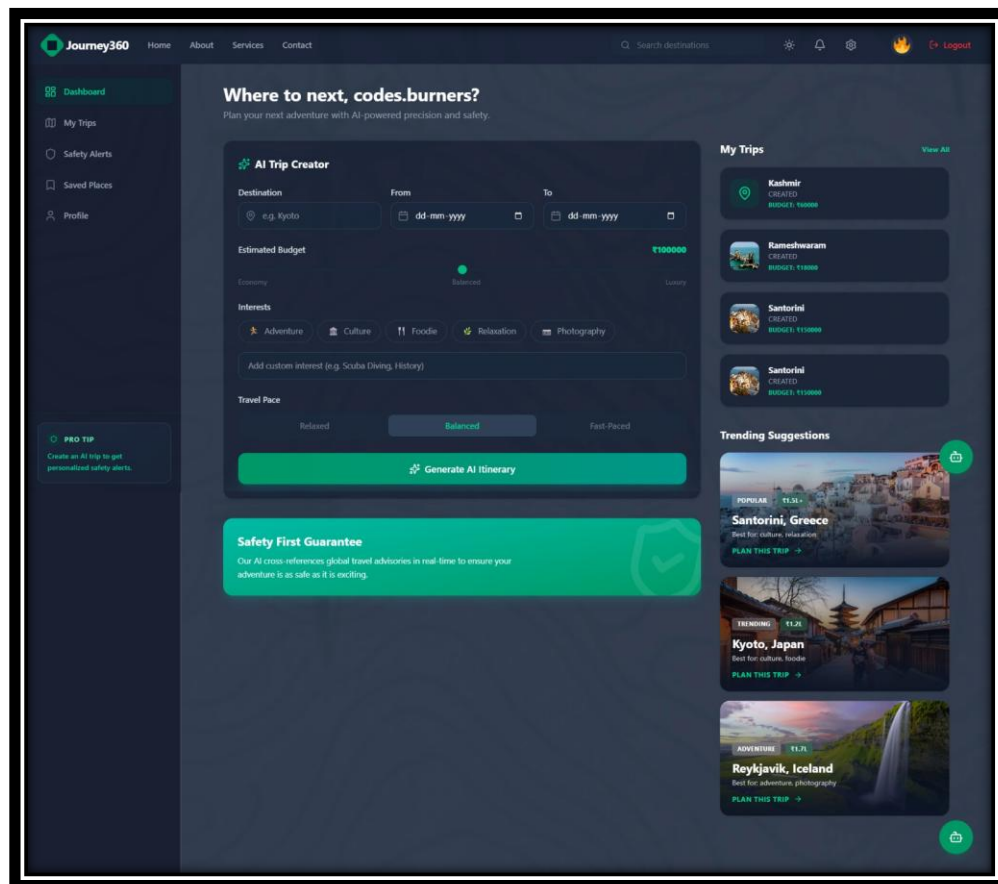
The prompt defined in backend/ai/itinerary.py positions the model as “*Journey360 AI, a precise travel consultant*” and enforces **hard constraints**, including:

1. Exact trip duration (no skipped or merged days)
2. Fixed daily structure (breakfast, activities, meals)
3. Realistic INR-based pricing
4. Mandatory latitude/longitude for every place
5. Safety advisories and weather context
6. Strict JSON-only output (no explanations or markdown)

The prompt also injects **local knowledge** and bans placeholder locations to ensure realism.

Human Control & Final Implementation:

AI output is never used directly. The backend validates structure, enforces business rules, recalculates costs, and ensures uniqueness and safety compliance. All orchestration was implemented using FastAPI, MongoDB, and React with Tailwind CSS. Route optimization was fully handled by the team using OpenRouteService and Google Directions.



4.2 Feature 2

4.2.1 Feature Name: Travel Assistant (AI Chat Interface)

4.2.2 Origin: Both (Self-Generated + AI-Assisted)

4.2.3 Description:

The Travel Assistant provides users with conversational, context-aware help during their trip—answering questions about their itinerary, nearby attractions, budget-friendly suggestions, and safety considerations.

AI Prompt Usage (Chat Assistant):

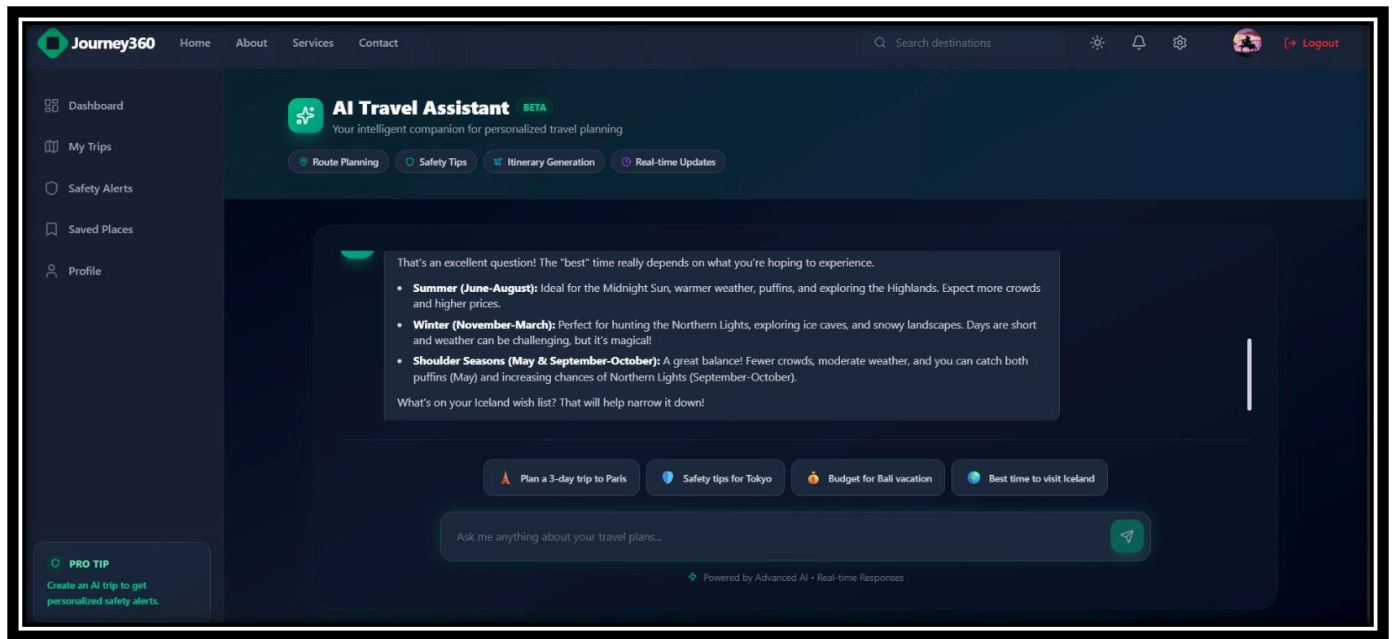
The prompt defined in `backend/ai/assistant.py` strictly limits AI behavior. It:

- Injects user context (destination, budget, interests)
- Enforces concise, friendly responses
- Explicitly prevents itinerary generation unless requested
- Acts as an advisory layer, not a planner

This ensures the assistant behaves like a helpful guide, not an uncontrolled generator.

Human Control & Final Implementation:

All prompt construction, response filtering, and UI presentation were implemented manually. The backend controls what context is passed to the AI, and responses are sanitized before being displayed. The interface was built using React, ensuring clarity and ease of use.



4.3 Feature 3

4.3.1 Feature Name: Real-Time Safety Center

4.3.2 Origin: Both (Self-Generated + AI-Assisted)

4.3.3 Description:

The Real-Time Safety Center helps travelers stay informed about risks at their destination by combining weather alerts, local news, and emergency access.

AI Prompt Usage (Safety Context Support):

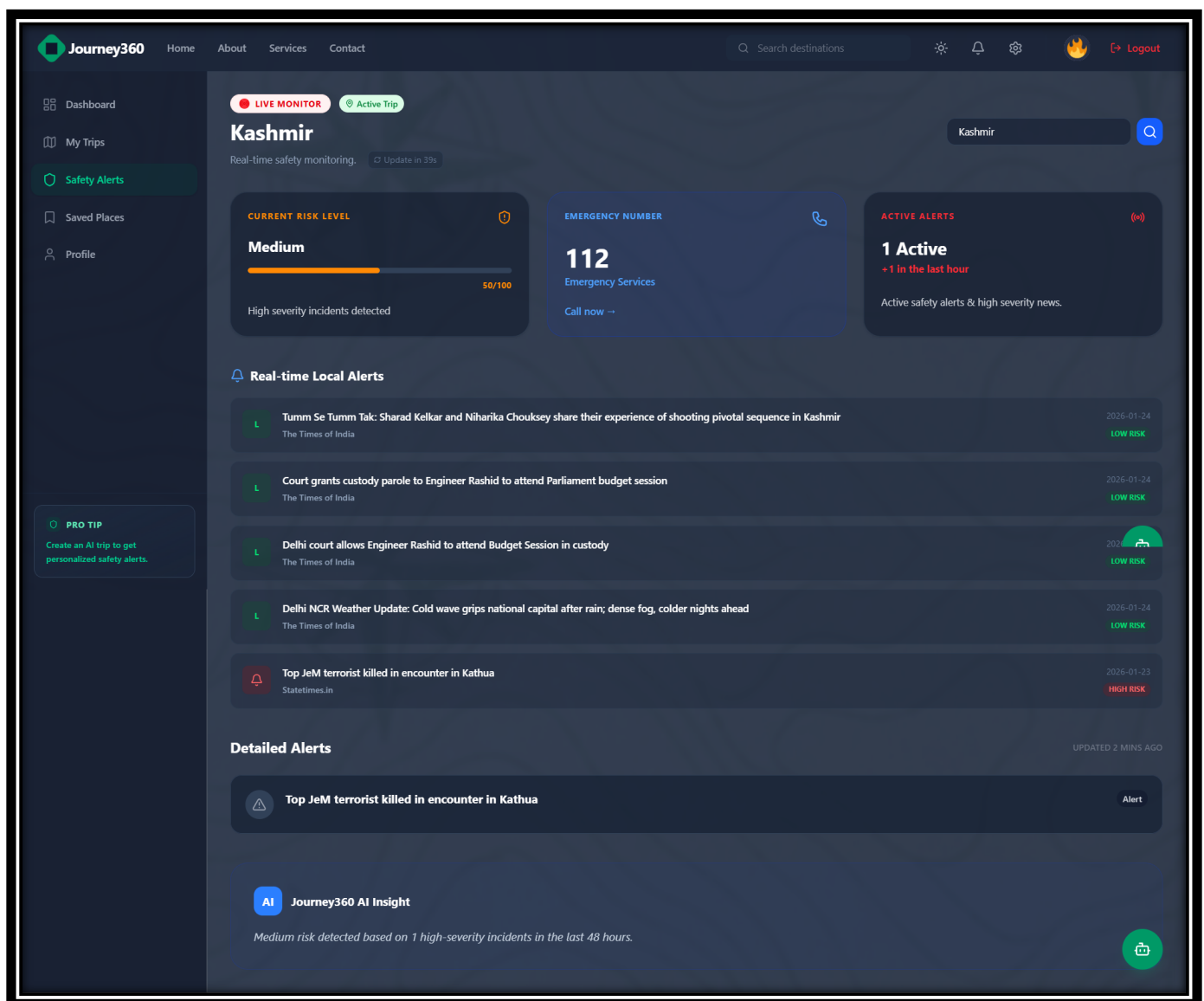
AI was used only at an exploratory level to understand how safety insights and advisories could be summarized clearly for users. Prompts focused on tone, clarity, and calm communication—especially for stressful scenarios.

Human Control & Final Implementation:

Actual safety logic is **entirely rule-based**, not AI-driven. The system uses:

- Weather data from OpenWeather API
- Location-based news via NewsAPI
- Keyword-based NLP severity detection (High / Medium / Low risk)

Risk scoring and thresholds were fully designed and implemented by the team using FastAPI and MongoDB. AI plays no role in decision-making or alert triggering.



4.4 Feature 4

4.4.1 Feature Name: Trip Management

4.4.2 Origin: Self-Generated

4.4.3 Description:

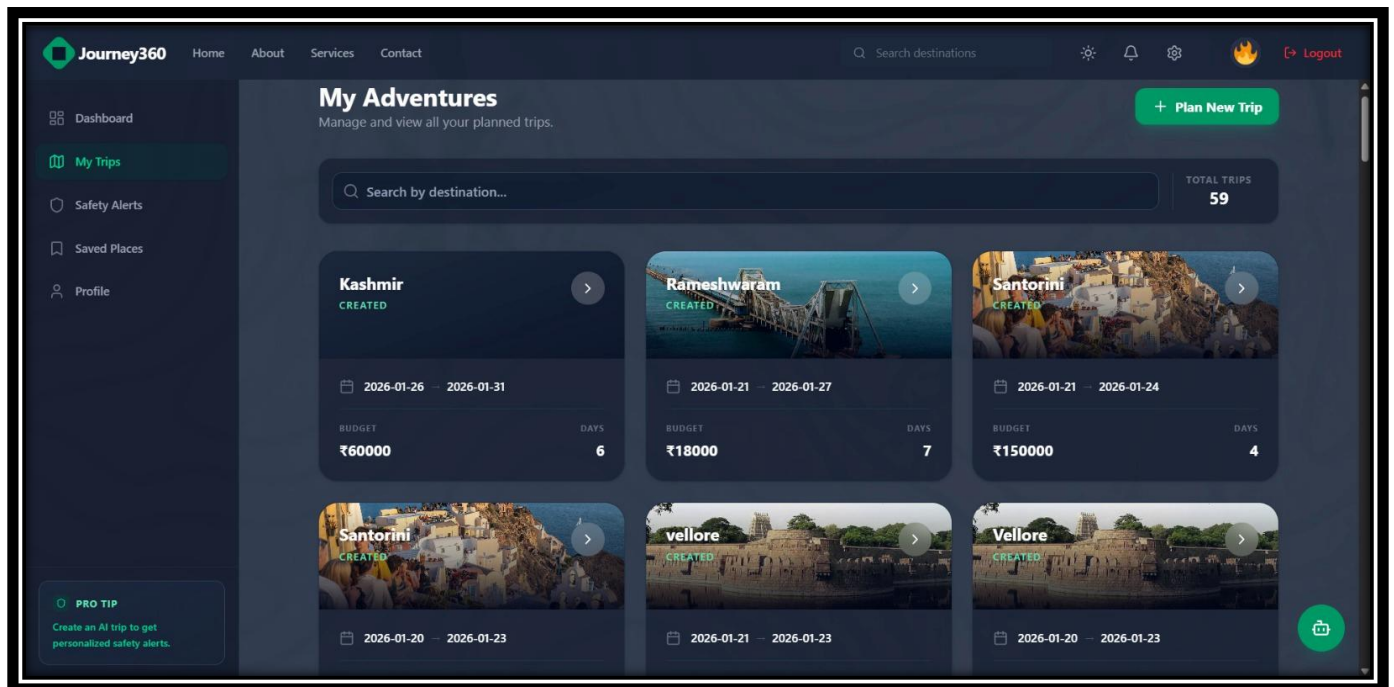
Trip Management allows users to view, manage, and verify trips, save favorite places, and review post-trip insights from a single dashboard.

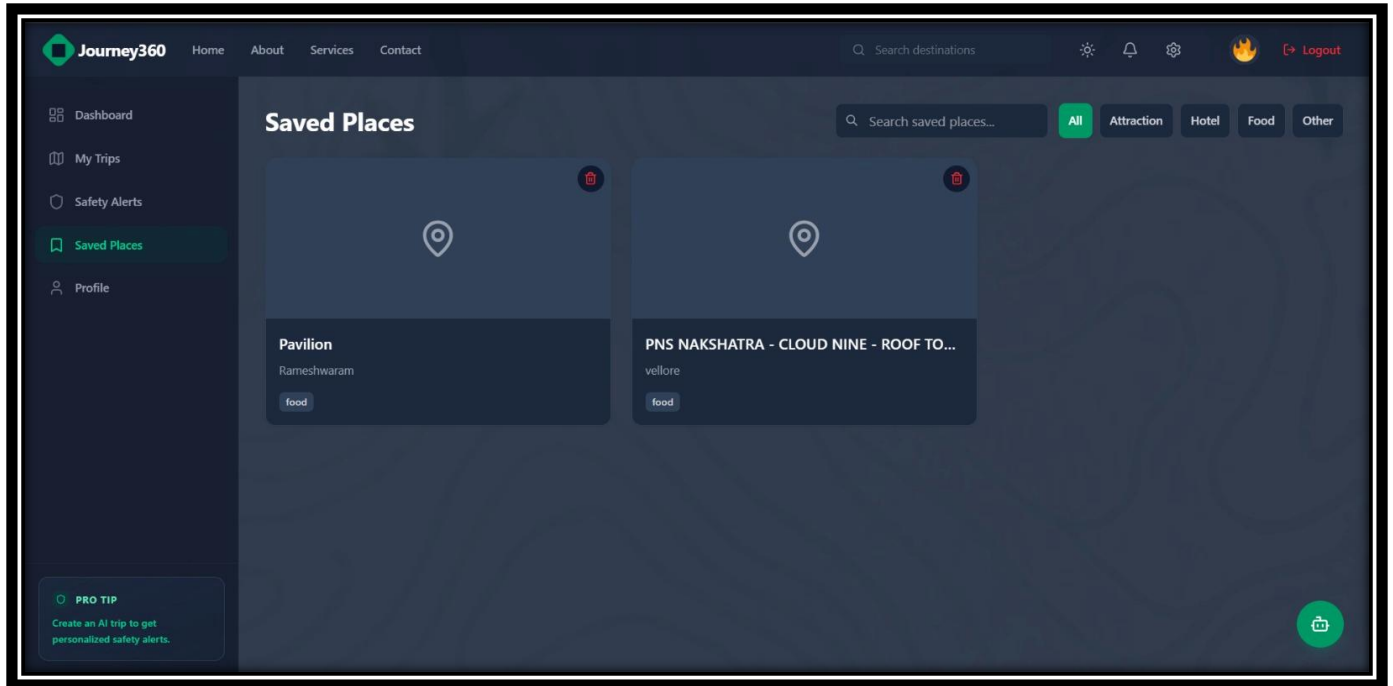
AI Usage:

No AI tools were used for this feature.

Human Control & Final Implementation:

All workflows, APIs, and UI components were fully designed and developed by the team using React, FastAPI, and MongoDB. This feature ensures a smooth transition across all phases of travel.





4.5 Feature 5

4.5.1 Feature Name: Authentication & User Profile Management

4.5.2 Origin: Self-Generated

4.5.3 Description:

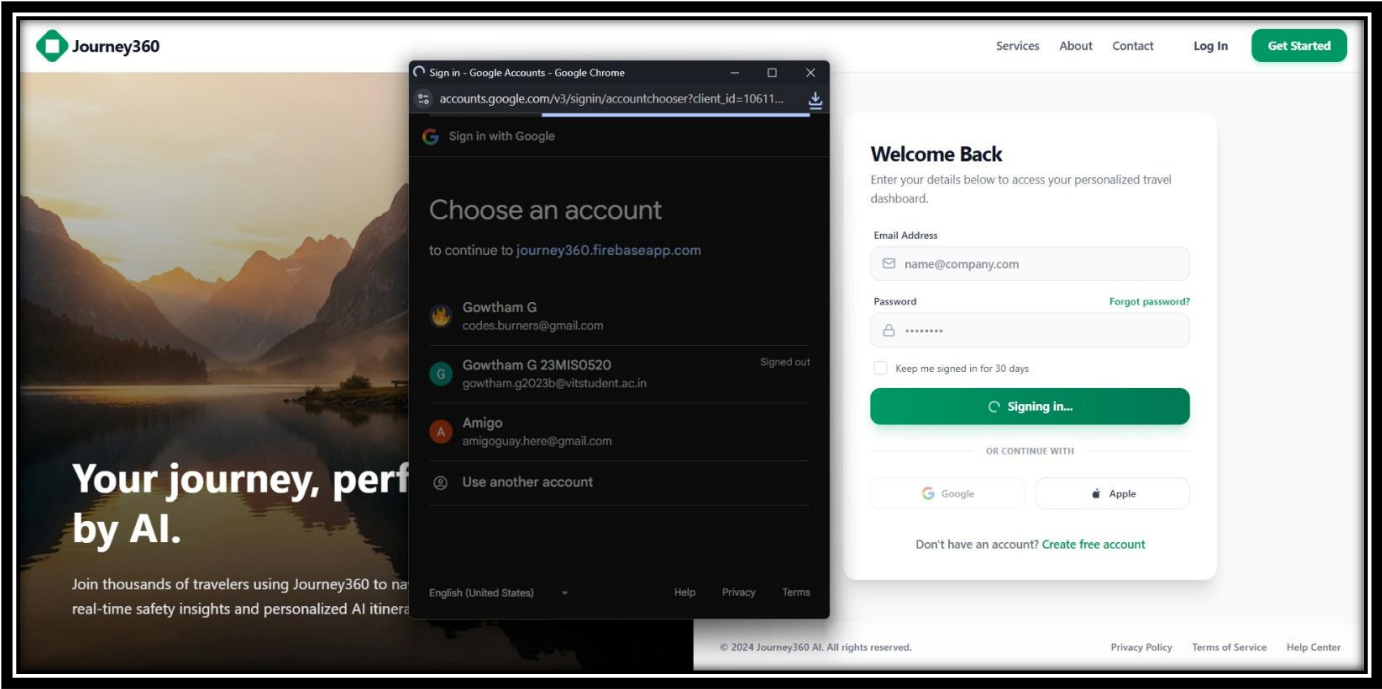
This feature ensures secure access and personalization for users through Email/Password login, Google Sign-In, and optional two-factor authentication.

AI Usage:

No AI tools were used.

Human Control & Final Implementation:

Authentication was implemented using Firebase Authentication. User preferences, profiles, and saved data were managed through backend APIs built with FastAPI and stored in MongoDB.



[Dashboard](#)[My Trips](#)[Safety Alerts](#)[Saved Places](#)[Profile](#)**PRO TIP**

Create an AI trip to get personalized safety alerts.

Settings

Manage your profile and preferences.

[Save Changes](#)

My Profile

Update your personal information.

LOGIN EMAIL (ACCOUNT ID)

* Your login email cannot be changed from this page.

FULL NAME

PROFILE PICTURE URL

BIO

PHONE NUMBER

DATE OF BIRTH



Preferences

Customize your regional usage.

CURRENCY

LANGUAGE

Dark Mode

I wear my eyes at night.



Notifications

Choose what we keep you updated on.

Safety Alerts

Critical alerts about your destination.



Itinerary Updates

Changes to your trip plans.



Email Reports

Weekly summaries and travel tips.



Security & Account

Protect your account.

Password

Last changed recently

[Change Password](#)

Two-Factor Authentication

Extra layer of security using Google Authenticator.

[Enabled](#)

Support

Need help?

[Contact Support Team](#)[Help Center & FAQs](#)

Danger Zone

Delete Account

This action cannot be undone.[Delete Account](#)[Sign Out](#)

Journey360 v2.2.0 (Build 5062)

4.6 Feature 6

4.6.1 Feature Name: AR Navigation

4.6.2 Origin: Both (Self-Generated + AI-Assisted)

4.6.3 Description:

AR Navigation overlays real-world directional guidance directly onto the camera feed, helping users navigate unfamiliar places intuitively.

AI Prompt Usage (Design Exploration Only):

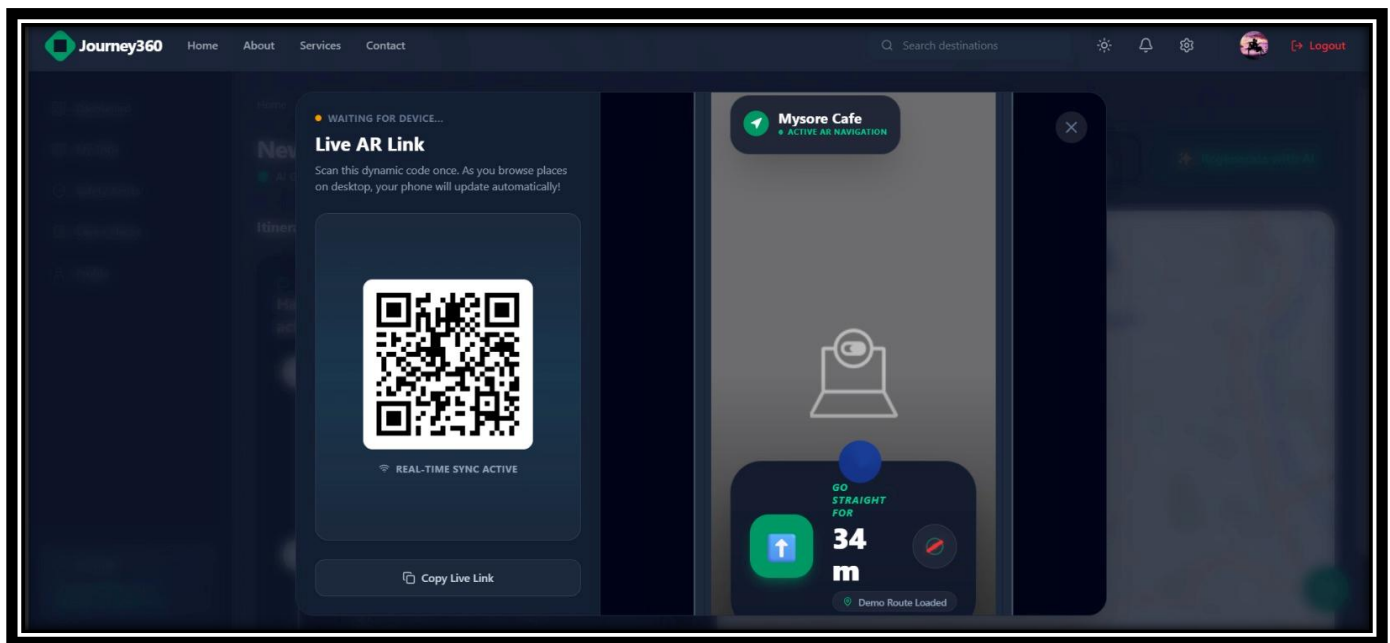
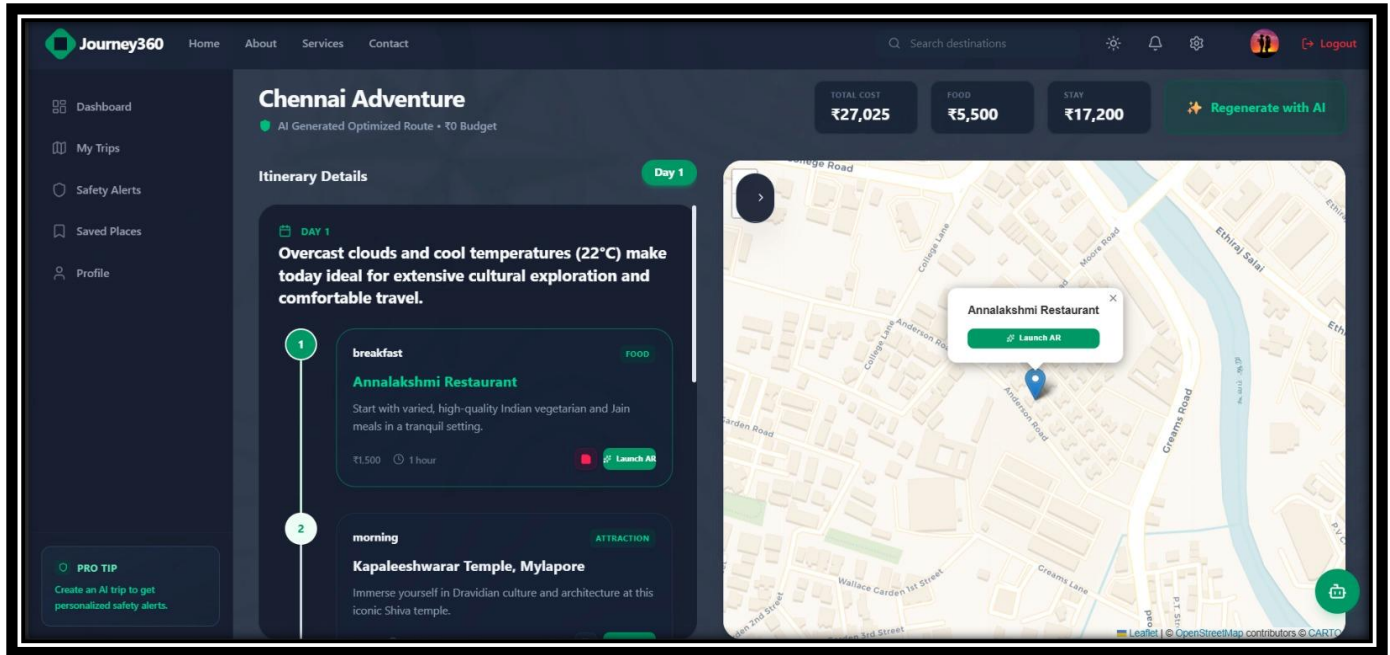
AI tools were used only during early design exploration to understand AR usability principles—specifically how to guide users clearly without visual overload. Prompts focused on clarity, safety, and minimal distraction.

Human Control & Final Implementation:

The final AR system is fully non-generative and algorithmic. It uses:

- Camera access via `navigator.mediaDevices.getUserMedia`
- GPS and device orientation APIs
- Route data from `OpenRouteService`
- Real-time bearing calculations
- Rendering via `Three.js`

All sensor fusion, navigation logic, rendering, and testing were implemented manually by the team.



4. Use Cases & User Flow

Journey360 was designed around real travel situations rather than isolated features. Below are the primary use cases that guided our system design.

Use Case 1: Trip Planning Before Travel

A user enters their destination, travel dates, budget, and interests. Journey360 generates a complete, day-wise itinerary including hotel suggestions, attractions, meals, route details, safety advisories, and estimated costs—all in one place.

This eliminates the need to manually search across multiple platforms and helps users start their journey feeling prepared and confident.

Use Case 2: On-Trip Navigation & Assistance

While exploring a destination, the user can access AR Navigation to receive real-time, camera-based guidance. The Travel Assistant provides contextual suggestions based on the user's current location, itinerary, and preferences.

This reduces confusion and makes navigation intuitive, especially in unfamiliar environments.

Use Case 3: Safety Awareness & Emergency Support

Journey360 continuously evaluates safety conditions using weather and local news data. Users receive safety insights and can trigger SOS assistance with a single tap if required.

This ensures safety is proactive rather than reactive.

Use Case 4: Post-Trip Review & Memory Capture

After the trip, users can review their journey, saved places, and AI-generated trip summaries. This adds value beyond the travel itself and improves long-term engagement.

5. External APIs & Tool Dependencies

Journey360 integrates multiple trusted external services to ensure real-world feasibility.

7.1 AI & Intelligence

- Google Generative AI (Gemini): itinerary generation, chat assistance, summaries

7.2 Maps & Navigation

- OpenRouteService: routing and directions
- Leaflet: interactive maps

7.3 Safety & Environment

- OpenWeather API: weather alerts
- News API: local safety news aggregation

7.4 Authentication & Security

- Firebase Authentication: Email/Password, Google Sign-In, optional 2FA

7.5 Device & Browser APIs

- Geolocation API
- Device Orientation API
- Camera access via MediaDevices

6. Impact & Value

Journey360 delivers impact at both the **user experience** and **system design** levels.

9.1 User Impact

- Reduces app switching and mental overload
- Increases confidence while navigating unfamiliar places
- Improves situational awareness through safety insights
- Saves planning time with structured itineraries

9.2 Safety Impact

- Proactive risk awareness instead of reactive alerts
- Calm, understandable safety communication
- Easy access to emergency features

9.3 Technical Impact

- Demonstrates responsible AI integration
- Shows real-world AR navigation feasibility
- Scalable architecture for future expansion

7. Challenges & Limitations

While developing Journey360, we encountered several challenges:

- **AR Accuracy:** Web-based AR depends on device sensors, which can vary in accuracy across devices
- **Real-Time Data Reliability:** Safety insights depend on third-party APIs, which may have delays or coverage gaps
- **Offline Constraints:** Full offline support is limited due to dependency on real-time data sources

- **AI Output Control:** Ensuring predictable AI behavior required strict prompt design and backend validation

These limitations informed our design decisions and highlighted areas for future improvement.

8. Key Learnings

Through Journey360, our team gained valuable technical and design insights:

- AI must be **governed, not trusted blindly**
- User safety requires **deterministic logic**, not probabilistic outputs
- AR usability depends more on clarity than visual complexity
- System design matters more than individual features
- Real-world problems demand **end-to-end thinking**, not isolated solutions

This project strengthened our understanding of building responsible, user-centric systems using modern web and AI technologies.

9. Closing Note

Across Journey360, AI is used as a **governed assistive layer**, never as an autonomous decision-maker. All prompts are tightly constrained, outputs are validated, and final behavior is controlled by deterministic backend and frontend logic. This approach ensures safety, predictability, realism, and user trust—especially in high-impact travel scenarios.

10. Ethical & Compliance Confirmation

AI usage complies with guidelines and policies. **Yes**

No proprietary or copyrighted data misused. **I Agree**

11. Declaration & Sign-Off

Name of Team Representative: Gowtham G

Role: Team Lead – Full stack Developer

Signature: Gowtham G

Date: 27.01.2026

Demo Video Link :

<https://drive.google.com/file/d/1hsSS44koLNLs8VfFXHsbumZIErhynIS/view?usp=sharing>

12.Team Contribution

Team Member	Role / Area	Key Contributions
Gowtham G	Backend & Trip Intelligence	Built the Smart Itinerary Builder, backend APIs, map integration, email-based itinerary delivery, and ensured smooth frontend-backend integration.
Aaron Alphons Thomas	Safety & AR Development	Developed the Safety Alert system with risk scoring and news integration, and improved AR Navigation through testing, debugging, and refinements.
M Kaarthik	UI Development & AR Support	Created the core UI structure, built the AI Assistant page, enhanced UI design, and supported AR feature integration.
Sandhiya U	Frontend & Documentation	Co-designed frontend interfaces, implemented safety and translation pages, and handled project documentation